CCF-CSP 前三题备考讲义

基础实现　常用算法　综合模拟

2026 年 10 月 9 日

使用说明

本讲义面向已经学过变量、分支、循环、数组和函数，希望系统训练 CCF-CSP 认证前三题的学生。学习主线是：先把短程序写准确，再学习减少重复计算的方法，最后把长题意拆成可靠的程序模块。

正文中的教学例题均给出完整的任务、输入输出、规模限制、样例、解答和参考实现，可以脱离外部网站学习。例题是围绕原资料知识点重新组织的教学题，**不等同于所链接平台题目的完整原版题面**。原题链接用于进一步练习，不能直接把教学例题的代码提交到规则不同的原题。

每章建议按三个步骤学习：先自己推演例题，再解释参考代码中的关键不变量，最后遮住答案完成章末练习。不会做时优先寻找“状态究竟表示什么”“哪一步重复计算”“哪个边界没有定义”，而不是立即记忆模板。

参考程序使用 C++17。为减少重复，使用竞赛环境常见的 `bits/stdc++.h`；该头文件属于 GNU 实现。其他编译器可换成程序实际使用的标准头文件，如 iostream、vector、algorithm、map、set、queue、sstream、string、limits、stdexcept。学习时使用 GNU C++17 编译即可，具体认证提交环境应在考试前核对。

第 1–10 章构成基础算法主线，第 11 章包含五个完整模拟案例，第 12 章提供带答案的综合训练。附录补充多重背包、多维背包、背包撤销等原资料中的提高内容。高级内容不影响先完成基础与模拟主线。

一个始终要回答的问题

一段代码通过样例，只说明它在那个输入上表现正确。每次写完都应检查：最小规模、最大规模、重复值、边界位置、不可达状态，以及题面允许但样例没有出现的情况。

目录

1　备考方法与程序基本功　5

1.1　从题面到程序的五个问题　. 5

1.2　复杂度决定能否完成计算　. 5

1.3　数值范围与输入边界　. 5

1.4　例题统计一组整数　. 6

1.5 练习与解答 6

2 基础枚举与小模拟 **7**

2.1 模拟的核心是状态时刻 7

2.2 例题网格的一轮同步更新 7

2.3 矩形相交的边界 8

2.4 日期模拟 8

2.5 练习与解答 9

3 排序与常用 STL 工具 **9**

3.1 按需要选择容器 9

3.2 例题按成绩和编号排序 10

3.3 去重与离散化 10

3.4 查找和删除的细节 11

3.5 练习与解答 11

4 字符串处理与解析基础 **11**

4.1 常用操作与区间含义 11

4.2 例题带引号的单行分词 12

4.3 C 字符数组与 string 的区别 12

4.4 解析与业务处理分开 13

4.5 练习与解答 13

5 前缀和与差分 **13**

5.1 一维前缀和的推导 13

5.2 例题静态区间求和 13

5.3 二维前缀和与容斥 14

5.4 差分把区间修改变成端点修改 14

5.5 例题批量区间加法 15

5.6 二维差分的四个角 15

5.7 练习与解答 15

6 二分与双指针 **16**

6.1 二分依赖单调性 16

6.2 例题切木条的最大整数长度 16

6.3 跳石头中的判定建模 17

6.4 双指针如何减少重复枚举 17

6.5 例题乘积小于 k 的连续区间个数 17
6.6 差值数对与重复元素 18
6.7 练习与解答 18

7 贪心与基础数学 **19**

7.1 贪心需要证明 19
7.2 例题合并石堆的最小代价 19
7.3 中位数最小化绝对距离和 20
7.4 质因数分解与溢出检查 20
7.5 矩阵运算先看维度 20
7.6 练习与解答 20

8 搜索与剪枝 **21**

8.1 搜索状态和递归职责 21
8.2 例题按字典序输出全排列 21
8.3 剪枝必须有排除理由 21
8.4 例题迷宫最短步数 22
8.5 访问标记的三种含义 22
8.6 练习与解答 23

9 动态规划入门 **23**

9.1 先定义状态再写转移 23
9.2 例题最长严格上升子序列 23
9.3 例题非空最大子段和 24
9.4 例题数字三角形的最大路径和 24
9.5 二维 DP 与最长公共子序列 25
9.6 记忆化搜索 25
9.7 练习与解答 26

10 基础背包与计数 **26**

10.1 先确定问题问的是什么 26
10.2 例题每件物品最多选一次 26
10.3 例题每种物品可以无限选择 27
10.4 用最小反例检查循环方向 28
10.5 例题无限硬币的组合数 28
10.6 例题每组最多选一件 29

10.7 恰好装满时如何处理不可达 30
10.8 练习与解答 30

11 第三题综合模拟专题 30

11.1 长题意的实现流程 30
11.2 例题路径规范化 31
11.3 例题授权和撤销的状态管理 32
11.4 例题带括号的整数表达式 33
11.5 例题树形文档的后代选择查询 36
11.6 例题模板展开与循环依赖 37
11.7 练习与解答 39

12 综合训练与复盘 40

12.1 练习方式 40
12.2 例题训练 A 差值数对 40
12.3 例题训练 B 矩形增量的最大值 40
12.4 例题训练 C 非抢占任务调度 41
12.5 如何复盘一次错误 43
12.6 提交前检查表 43

A 提高专题与模型扩展 44

A.1 多重背包与二进制拆分 44
A.2 混合背包与多维费用 44
A.3 方案恢复 45
A.4 计数背包的撤销 45
A.5 LIS 优化与排列 LCS 45
A.6 实数二分与三分的前提 46
A.7 可删堆的延迟删除 46
A.8 字符串匹配的线性预处理 46
A.9 附录练习与解答 47

B 资料来源与继续练习 47

1 备考方法与程序基本功

1.1 从题面到程序的五个问题

读题后先写下：输入包含什么；输出要求什么；一个对象需要哪些属性；一次操作如何改变状态；数据规模允许多少工作量。例如，统计学生成绩时，对象可以是“编号与成绩”，状态是所有学生记录，输出可能是排序结果或某种统计值。不要把没有读懂的规则直接埋进条件分支。

遇到长题意，先处理名词和动词。名词通常对应结构体、数组或映射；动词通常对应函数。对“先……再……”“同时……”“若不存在……”等词逐一标记，它们常决定程序是否正确。

1.2 复杂度决定能否完成计算

设输入规模为 n。一次遍历是 $O(n)$，两层独立遍历通常是 $O(n^2)$；每次将区间减半的查找是 $O(\log n)$。排序通常需要 $O(n \log n)$ 比较。复杂度表示增长趋势，不是精确运行秒数。

若 $n = 2 \times 10^5$，两重枚举约需要 4×10^{10} 次配对，通常不可行；排序的比较数量级约为几百万，更值得考虑。若 $n \le 20$，枚举全部子集有 $2^{20} = 1048576$ 种，可能合适。实际限制还与常数、内存、实现和时限有关，不能把某个次数阈值当作保证。

操作	典型复杂度	常见改进方向
每次重新求区间和	$O(nq)$	静态数组用前缀和降为 $O(n+q)$。
枚举所有数对	$O(n^2)$	利用排序、映射或双指针。
每个状态重复递归	可能指数级	完整状态可缓存时使用记忆化。
每次扫描取最小值	每次 $O(n)$	动态维护可用优先队列。
每个答案都尝试	与答案范围成正比	判定具有单调性时考虑二分。

1.3 数值范围与输入边界

常见 32 位有符号整数的最大值约为 2.1×10^9。若 $n = 2 \times 10^5$ 且每项绝对值不超过 10^9，总和可能达到 2×10^{14}，需要 long long。尤其注意：两个 int 先相乘再赋给 long long，乘法仍可能先溢出。应写 `1LL*a*b`，让乘法从一开始就在更宽的类型中进行。

字符串用 `cin >> s` 读取时遇到空白结束；`getline(cin,s)` 读取整行。在先读取数字、再读取整行时，要处理数字后剩余的换行。可先调用一次 getline 消耗该行剩余内容。若前导空格是有效信息，不要使用会吞掉它们的处理方式。

统一区间含义也很重要。数组前缀和常采用 $[l,r]$ 闭区间；STL 的迭代器区间通常采用 $[first, last)$，last 指向最后一个元素之后。两种习惯都可以使用，但转换处必须明确。

1.4 例题统计一组整数

题目。输入 n 个整数，输出总和、最小值、最大值，以及出现次数最多的值及其次数。若多个值并列，输出最小的值。$1 \le n \le 2 \times 10^5$，$|a_i| \le 10^9$。

输入。第一行 n，第二行 n 个整数。**输出**。第一行依次输出总和、最小值、最大值；第二行输出众数及出现次数。

```
输入
6
3 -1 3 2 -1 5
输出
11 -1 5
-1 2
```

解答。总和与极值可边读边更新。用 map 保存 “值到次数” 的关系。map 按键升序遍历，仅在次数严格更大时更新答案，就能保留并列时较小的值。最小值不能初始化成 0，否则全为正数时会出错；最大值同理。复杂度 $O(n \log n)$，空间 $O(n)$。

完整参考实现 `01_stats.cpp`

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n; cin >> n;
    long long sum = 0, mn = LLONG_MAX, mx = LLONG_MIN;
    map<long long, int> cnt;
    for (int i = 0; i < n; ++i) {
        long long x; cin >> x;
        sum += x; mn = min(mn, x); mx = max(mx, x); ++cnt[x];
    }
    long long mode = 0; int best = 0;
    for (auto [x, c] : cnt) {
        if (c > best) { best = c; mode = x; }
    }
    cout << sum << ' ' << mn << ' ' << mx << '\n';
    cout << mode << ' ' << best << '\n';
}
```

1.5 练习与解答

练习 1。数组长度为 10^5，每项不超过 10^9，总和是否可以用 int？**解答**。最大总和为 10^{14}，应使用 long long。若单项也可能相乘，还要检查乘法是否提前提升类型。

练习 2。对 $[4,4,4]$，上题输出什么？**解答**。第一行是 `12 4 4`，第二行是 `4 3`。这检查重复值与极值更新。

练习 3。有 10^5 次查询，每次扫描 10^5 个数求和，工作量约是多少？**解答**。约 10^{10} 次元素访问。若数组不变，应预处理前缀和；若存在修改，则先判断能否离线处理，不直接套静态前缀和。

2 基础枚举与小模拟

2.1 模拟的核心是状态时刻

模拟题要求按规则执行，并不意味着随意堆条件。先定义变量表示“操作之前”还是“操作之后”，再明确更新顺序。若同一轮的所有变化依赖旧状态，应把旧状态和新状态分开。若操作依次发生，则后面的操作通常应看到前面操作的结果。

枚举题还需说明为什么不重不漏。例如枚举无序数对，使用 $1 \le i < j \le n$，每对只出现一次；若使用 $i \ne j$，则每对通常出现两次。判断条件正确和枚举范围正确同样重要。

2.2 例题网格的一轮同步更新

题目。给定一个 $n \times m$ 的 0/1 网格。一次更新后，每个格子的新值由其旧状态的上下左右四邻格决定：值为 1 的邻格至少有两个，则新值为 1，否则为 0。越界邻格不计入。只进行一轮更新，$1 \le n, m \le 500$。

输入。第一行 n, m，随后每行 m 个 0 或 1。**输出**。更新后的网格。

输入

```
2 3
1 0 1
0 1 0
```

输出

```
0 1 0
1 0 1
```

推演。左上格的两个有效邻格都是 0，所以变为 0；上方中间格的三个邻格都是 1，所以变为 1。所有计数都使用原数组 a，把答案放进 b。若直接覆盖 a，后面的格子会读到已经变化的数据。

正确性。程序对每格恰好检查四个方向，只有合法坐标参与计数，因此统计值与题意一致；写入 b 不改变 a，满足同步更新。时间和空间均为 $O(nm)$。

完整参考实现 02_grid.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m; cin >> n >> m;
    vector<vector<int>> a(n, vector<int>(m)), b(n, vector<int>(m));
    for (auto &row : a) for (int &x : row) cin >> x;
    int dx[] = {-1, 1, 0, 0}, dy[] = {0, 0, -1, 1};
    for (int i = 0; i < n; ++i) for (int j = 0; j < m; ++j) {
        int c = 0;
        for (int k = 0; k < 4; ++k) {
            int x = i + dx[k], y = j + dy[k];
            if (x >= 0 && x < n && y >= 0 && y < m) c += a[x][y];
        }
        b[i][j] = (c >= 2);
    }
    for (auto &row : b) {
        for (int j = 0; j < m; ++j) cout << row[j] << (j+1 == m ? '\n' : ' ');
    }
}
```

2.3 矩形相交的边界

两个与坐标轴平行的矩形分别为 $[x_1, x_2] \times [y_1, y_2]$ 和 $[u_1, u_2] \times [v_1, v_2]$，且各端点已经有序。相交宽度与高度为

$$w = \max(0, \min(x_2, u_2) - \max(x_1, u_1)), \qquad h = \max(0, \min(y_2, v_2) - \max(y_1, v_1)).$$

面积为 wh。例如 $[0,3] \times [0,2]$ 与 $[2,5] \times [1,4]$ 的交集宽、高都为 1，面积为 1。仅接触边界时面积为 0。若题目统计整数格点而非连续面积，端点规则不同，不能擅自加 1 或减 1。

2.4 日期模拟

日期问题先确定历法。公历闰年的判定是：能被 400 整除，或者能被 4 整除但不能被 100 整除。月份长度用表保存，二月按闰年调整。把日期加一天时，先增加日；若超过当月天数，日改为 1 并增加月；若月变成 13，再进入下一年。每步只有一个明确定义的状态变化。

```cpp
bool leap(int y) {
    return y % 400 == 0 || (y % 4 == 0 && y % 100 != 0);
}
int days(int y, int m) {
    int d[] = {0,31,28,31,30,31,30,31,31,30,31,30,31};
    return d[m] + (m == 2 && leap(y));
}
```

2.5 练习与解答

练习 1。对只有一个格子的网格，原值为 1，更新后是多少？**解答**。为 0，因为没有有效邻格。它检查越界和最小规模。

练习 2。2000 年和 2100 年是否闰年？**解答**。2000 能被 400 整除，是闰年；2100 能被 100 整除但不能被 400 整除，不是闰年。因此两年的二月分别有 29 天和 28 天。

练习 3。矩形 $[0,2]\times[0,2]$ 与 $[2,4]\times[0,2]$ 的相交面积是多少？**解答**。宽度为 0，面积为 0；边接触不产生正面积。

练习 4。若网格更新两轮，是否可以一直只计算 b？**解答**。不可以。第一轮结束后应交换 a、b，使第二轮从第一轮结果出发；每轮仍必须完整覆盖 b 的全部位置。

3 排序与常用 STL 工具

3.1 按需要选择容器

工具	适用需求	关键注意事项
vector	可变长顺序表，下标访问	中间插入删除通常为 $O(n)$；扩容可能使迭代器失效。
map	根据键查找和统计，有序遍历	查找、插入通常为 $O(\log n)$；下标访问不存在键会插入默认值。
set / multiset	动态有序集合	set 去重；multiset 可重复。按值 erase 可能删掉全部相等元素。
queue	先进先出	front 读队首，pop 只删除，不返回元素。
stack	后进先出	top 读栈顶，空栈不能访问 top。
priority_queue	动态取最大或最小	默认大根堆；不支持任意位置直接删除。
sort	对可随机访问区间排序	比较器要满足严格弱序，不能把小于写成小于等于。
lower_bound	有序序列中第一个不小于 x 的位置	可能返回 end，访问前应检查。
upper_bound	有序序列中第一个大于 x 的位置	与 lower_bound 之差可统计出现次数。

3.2 例题按成绩和编号排序

题目。输入 n 名学生的编号和成绩，按成绩降序排列；成绩相同按编号字典序升序。编号为不含空格的字符串且互不相同，$1 \le n \le 2 \times 10^5$，成绩在 0 到 100 之间。输入第一行 n，随后每行编号、成绩；输出排好序的记录。

输入
4
b02 90
a01 95
a02 90
a03 90
输出
a01 95
a02 90
a03 90
b02 90

解答。比较器先比较主关键字，只有主关键字相等才比较次关键字。对于完全相同的对象，比较结果应为 false。使用非严格比较可能破坏排序算法的前提。时间 $O(n \log n)$，存储记录需要 $O(n)$ 空间。

完整参考实现 03_rank.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
struct Student { string id; int score; };
int main() {
    int n; cin >> n;
    vector<Student> a(n);
    for (auto &x : a) cin >> x.id >> x.score;
    sort(a.begin(), a.end(), [](const Student &x, const Student &y) {
        if (x.score != y.score) return x.score > y.score;
        return x.id < y.id;
    });
    for (auto x : a) cout << x.id << ' ' << x.score << '\n';
}
```

3.3 去重与离散化

离散化把一组值替换成它们在去重有序集合中的排名，保留相等关系与大小关系，但**不保留原来的距离**。例如原值 $[100, 5, 100, 20]$ 排序去重后是 $[5, 20, 100]$，映射成 $[3, 1, 3, 2]$。若问题涉及长度或面积，仍要保留原坐标。

```cpp
vector<long long> keys = a;
sort(keys.begin(), keys.end());
keys.erase(unique(keys.begin(), keys.end()), keys.end());
vector<int> rank(a.size());
for (size_t i = 0; i < a.size(); ++i)
    rank[i] = lower_bound(keys.begin(), keys.end(), a[i]) - keys.begin() + 1;
```

unique 处理相邻的等价元素并返回新的逻辑末尾，容器长度由 erase 真正改变。排序不是 unique 的硬性前提，但若希望全局去重，必须让相同值相邻或使用其他去重方案。

3.4 查找和删除的细节

只想查询键是否存在时，使用 find，避免 `mp[key]` 隐式创建键。在 set 中找第一个不小于 x 的元素，优先调用 `s.lower_bound(x)`；对树容器迭代器调用通用 lower_bound 可能产生线性数量的迭代器移动。

```cpp
auto it = s.lower_bound(x);
if (it != s.end()) cout << *it;       // successor, possibly equal
if (it != s.begin()) cout << *prev(it); // strict predecessor
auto p = ms.find(x);
if (p != ms.end()) ms.erase(p);      // erase one occurrence
```

3.5 练习与解答

练习 1。为什么比较器 `return x.score >= y.score;` 不正确？**解答**。对象与自己比较会得到 true，违反严格性；并列对象互相都“小于”对方，排序前提被破坏。

练习 2。数组 $[2,2,5,7]$ 中 x=2 的 lower_bound 和 upper_bound 的 0 下标分别是什么？**解答**。分别是 0 和 2，差为 2，表示两个 2。x=8 时两者都返回 end，即下标 4，不能解引用。

练习 3。把坐标 $[1,100,101]$ 离散为 $[1,2,3]$，原区间长度是否仍可由排名差计算？**解答**。不可以。原来的前两点相距 99，排名差只有 1，离散化只保序。

4 字符串处理与解析基础

4.1 常用操作与区间含义

string 的下标从 0 开始。`s.substr(pos,len)` 的第二个参数是长度；`s.find(t)` 返回首次匹配位置，找不到时返回 `string::npos`。不要把 npos 随意存进 int 后再与某个猜测值比较。字符串比较是字典序，例如 `"10" < "2"`，与整数大小不同。

扫描字符串时最好约定：“指针 p 总是指向下一个尚未处理的字符”。每个分支要么推进 p，要么返回。若某个分支既不推进也不退出，就容易死循环。处理完整段落时，通常比反复在字符串中间 erase 更清楚、也更高效。

4.2 例题带引号的单行分词

题目。一行文本由普通字符、空格和英文双引号组成。引号外的连续空格分隔单词；引号内的空格属于单词；双引号本身不输出。相邻的普通片段和引号片段属于同一个单词。空引号表示一个空单词。本题不支持转义字符。若引号不匹配，输出 ERROR。行长不超过 10^5。

输入。一整行文本，可为空。**输出**。合法时先输出单词数，随后每个单词单独一行，用方括号包住，便于看见空单词和空格。

```
输入
add "hello world" "" x
输出
4
[add]
[hello world]
[]
[x]
```

状态设计。quoted 表示当前是否在引号内；cur 保存当前单词；active 表示当前单词已经开始。active 不能由 cur 是否为空代替，否则空引号会丢失。遇到引号切换 quoted；遇到引号外空格时结束当前单词；其余字符加入 cur。扫描结束后检查引号是否闭合，再处理最后一个单词。

正确性。每个字符恰好进入一种分支。引号仅改变状态，分隔空格仅结束一个已开始的单词，因此保留的字符、单词边界和空单词均符合规则。时间 $O(L)$，输出存储 $O(L)$。

完整参考实现 04_tokens.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s; getline(cin, s);
    vector<string> words; string cur;
    bool quoted = false, active = false;
    for (char c : s) {
        if (c == '"') { quoted = !quoted; active = true; }
        else if (c == ' ' && !quoted) {
            if (active) { words.push_back(cur); cur.clear(); active = false; }
        } else { cur += c; active = true; }
    }
    if (quoted) { cout << "ERROR\n"; return 0; }
    if (active) words.push_back(cur);
    cout << words.size() << '\n';
    for (const auto &w : words) cout << '[' << w << "]\n";
}
```

4.3 C 字符数组与 string 的区别

C 字符串以终止字符 '\0' 结尾，数组需要额外空间存放它。strlen 统计终止符前的字符数。定义后的字符数组不能用赋值表达式整体赋值；string 可以直接赋值。strcmp 返回值的符号表示字典

序关系，不保证只返回 1 或-1。字符分类函数如 isdigit 接收字符时，可先转为 unsigned char，避免负 char 值带来的未定义行为。

4.4 解析与业务处理分开

例如命令 `GRANT alice file1`，解析层先产生三个字符串，再由执行层决定如何修改权限。这样出现错误时，可以先打印解析结果，判断错在读入还是状态更新。第 11 章会进一步用函数处理路径和表达式。

4.5 练习与解答

练习 1。输入 `a"b c"d`，分词结果是什么？**解答**。只有一个单词 `ab cd`。相邻片段没有引号外空格，不应拆开。

练习 2。输入只有三个空格，输出什么？**解答**。单词数为 0。输入 `""` 时则是一个空单词，输出 1 和 `[]`。

练习 3。若题目新增反斜杠转义，当前程序能直接使用吗？**解答**。不能。需要新增 escape 状态：读取反斜杠后，下一个字符按普通字符处理，不切换引号状态。还要定义行末孤立反斜杠是否非法。规则改变后必须重新列出状态转移。

5 前缀和与差分

5.1 一维前缀和的推导

定义 $pre[0]=0$，$pre[i]=a_1+\cdots+a_i$。则

$$pre[i]=pre[i-1]+a_i,\qquad \sum_{i=l}^{r}a_i=pre[r]-pre[l-1].$$

原因是前一个总和包含 1 到 r，减去 1 到 $l-1$ 后只剩所需区间。预处理 $O(n)$，每次查询 $O(1)$。该方法适合**原数组不变**的多次查询。

5.2 例题静态区间求和

输入长度为 n 的数组和 q 次闭区间查询，每次输出区间和。$1\le n,q\le 2\times 10^5$，$|a_i|\le 10^9$，保证 $1\le l\le r\le n$。第一行 n,q，第二行数组，随后 q 行各给出 l,r。

输入

```
5 3
2 -1 3 4 5
1 3
2 4
5 5
```

输出

```
4
6
5
```

前缀数组为 $[0,2,1,4,8,13]$，查询 $[2,4]$ 得 $8-2=6$。0 号位置让 $l=1$ 的查询无需特殊判断。使用 long long 存前缀和。总复杂度 $O(n+q)$，空间 $O(n)$。

完整参考实现 05_prefix.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, q; cin >> n >> q;
    vector<long long> pre(n + 1);
    for (int i = 1; i <= n; ++i) { long long x; cin >> x; pre[i] = pre[i-1] + x; }
    while (q--) { int l, r; cin >> l >> r; cout << pre[r] - pre[l-1] << '\n'; }
}
```

5.3 二维前缀和与容斥

设 $S[i][j]$ 为从 $(1,1)$ 到 (i,j) 的矩形和，0 行和 0 列为 0：

$$S[i][j] = S[i-1][j] + S[i][j-1] - S[i-1][j-1] + a[i][j].$$

左上角被前两项算了两次，因此减一次。查询左上 (x_1,y_1)、右下 (x_2,y_2) 的矩形：

$$S[x_2][y_2] - S[x_1-1][y_2] - S[x_2][y_1-1] + S[x_1-1][y_1-1].$$

例如矩阵 $\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$ 的第 2–3 列总和是 $2+3+5+6=16$；用公式得到 $21-0-5+0=16$。预处理 $O(nm)$，每个查询 $O(1)$。

5.4 差分把区间修改变成端点修改

定义 $d[1]=a[1]$，$d[i]=a[i]-a[i-1]$。原数组是差分数组的前缀和。给 $[l,r]$ 每个元素加 v，只需

$$d[l] \mathrel{+}= v, \qquad d[r+1] \mathrel{-}= v.$$

前一个修改让从 l 开始的前缀和都增加 v，后一个修改从 r+1 起抵消。必须为 n+1 位置预留空间。该做法适合执行全部修改后再求最终结果。

5.5 例题批量区间加法

输入长度为 n 的数组，执行 q 次操作，每次给闭区间 $[l,r]$ 加 v，最后输出整个数组。$1 \le n,q \le 2\times 10^5$，初始值和修改值绝对值不超过 10^9。

输入
5 2
1 2 3 4 5
2 4 3
1 5 -1
输出
0 4 5 6 4

先用初始数组构造差分。所有修改都可叠加，最后做一次前缀累加。总时间 $O(n+q)$，空间 $O(n)$。上例第一轮得到 $[1,5,6,7,5]$，第二轮整体减 1。

完整参考实现 06_difference.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, q; cin >> n >> q;
    vector<long long> a(n+1), d(n+2);
    for (int i = 1; i <= n; ++i) { cin >> a[i]; d[i] = a[i] - a[i-1]; }
    while (q--) { int l, r; long long v; cin >> l >> r >> v; d[l] += v; d[r+1] -= v; }
    long long x = 0;
    for (int i = 1; i <= n; ++i) { x += d[i]; cout << x << (i == n ? '\n' : ' '); }
}
```

5.6 二维差分的四个角

给矩形 $[x_1,x_2]\times[y_1,y_2]$ 加 v：

```cpp
d[x1][y1] += v;
d[x2+1][y1] -= v;
d[x1][y2+1] -= v;
d[x2+1][y2+1] += v;
```

随后对 d 求二维前缀和，就得到每个格子的增量。左上角启动影响，下方和右方分别取消，右下角因取消两次再补一次。若有初始矩阵，可先恢复增量再加到初值上，或按 $d[i][j]=a[i][j]-a[i-1][j]-a[i][j-1]+a[i-1][j-1]$ 建初始差分。

5.7 练习与解答

练习 1。数组 $[3,-2,7]$ 的前缀和与差分是什么？**解答**。前缀和含 0 号为 $[0,3,1,8]$；差分为 $[3,-5,9]$。注意差分可以为负。

练习 2。在全 0 的 3×3 网格中，给左上 2×2 加 2，再给右下 2×2 加 3，结果是什么？**解答**。

$$\begin{pmatrix}2&2&0\\2&5&3\\0&3&3\end{pmatrix}.$$

中心格被覆盖两次，得到 5。按四角更新再恢复，应得到同样矩阵。

练习 3。修改和查询交替出现，是否仍能用上述普通差分每次 $O(1)$ 回答区间和？**解答**。不能。普通差分方便累计修改，但恢复全部值需要一次扫描。需要重新分析是否可离线，或使用树状数组等动态结构，不能隐瞒恢复成本。

6 二分与双指针

6.1 二分依赖单调性

二分的对象可以是有序数组，也可以是答案。核心是判定结果沿某个方向只变化一次。例如长度 x 能切出至少 k 段木头，那么更短的正整数长度也能做到；可行区间是前缀。先写出“真、真、假、假”的顺序，再选择搜索第一个真或最后一个真。

在闭区间 $[lo,hi]$ 搜索最后一个可行值时，取上中位数 $mid=lo+(hi-lo+1)/2$。若 mid 可行，令 lo=mid；否则 hi=mid-1。上中位数保证剩下两个元素时仍能缩小区间。每步至少排除一个候选，区间最终只剩答案。

6.2 例题切木条的最大整数长度

有 n 根木条，第 i 根长度为 a_i。每根可切成若干长度相同的整段，剩余部分丢弃，不能拼接不同木条。至少要得到 k 段，求最大正整数段长；若连长度 1 都无法满足，输出 0。$1\le n\le 2\times10^5$，$1\le a_i\le10^9$，$1\le k\le10^{12}$。输入第一行 n、k，第二行各长度。

输入

```
4 7
20 15 10 17
```

输出

```
7
```

长度 7 时可得 $2+2+1+2=7$ 段；长度 8 时只有 $2+1+1+2=6$ 段。判定函数为 $\sum_i\lfloor a_i/x\rfloor\ge k$，随 x 增大计数不增。0 作为“无正整数解”的哨兵，不传入判定函数，因此不会除以 0。复杂度 $O(n\log\max a_i)$，空间 $O(n)$。

完整参考实现 07_wood.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; long long k; cin >> n >> k;
    vector<long long> a(n);
    long long hi = 0;
    for (auto &x : a) { cin >> x; hi = max(hi, x); }
    auto ok = [&](long long len) {
        long long count = 0;
        for (long long x : a) { count += x / len; if (count >= k) return true; }
        return false;
    };
    long long lo = 0;
    while (lo < hi) {
        long long mid = lo + (hi - lo + 1) / 2;
        if (ok(mid)) lo = mid; else hi = mid - 1;
    }
    cout << lo << '\n';
}
```

6.3 跳石头中的判定建模

若目标是让相邻保留石头的最小距离尽量大，可以二分距离 x。判定时从起点向后扫描，若当前石头与上次保留位置距离小于 x，则至少需要移走其中一块。通常贪心优先移走当前中间石头，以保留更靠前的落点，为后续留下空间。计数的含义应是**达到距离要求至少需移走的数量**，不是最多能移走多少。

终点不可移走，需要单独验证末段，或者采用已证明等价的计数处理。不要只在中间石头上检查距离。二分答案的困难往往在判定函数的正确性，而不是二分循环本身。

6.4 双指针如何减少重复枚举

设右端点逐渐增加，若满足条件的最小左端点不会向左移动，就可保存它而不重复扫描。两个指针分别最多移动 n 次，总移动次数为 $O(n)$。有两层循环不代表一定是 $O(n^2)$，要统计指针在全程移动多少次。

6.5 例题乘积小于 k 的连续区间个数

给定 n 个**正整数**，统计乘积严格小于 k 的非空连续子数组数量。$1 \le n \le 2 \times 10^5$，$1 \le a_i \le 10^6$，$0 \le k \le 10^9$。输入第一行 n、k，第二行数组。

输入

```
4 100
10 5 2 6
```

输出

```
8
```

合法区间包括 4 个单元素、$[10,5]$、$[5,2]$、$[2,6]$、$[5,2,6]$，共 8 个。维护窗口乘积，每加入右端点，就在乘积过大时删除左端点。缩小后，以当前右端点结尾的合法区间有 $right - left + 1$ 个。

所有数至少为 1，因此缩短区间不会使乘积增大，合法左端点形成后缀。若允许负数，这个性质会失效；若允许 0，除法维护也需另外处理。k 不大于 1 时没有合法非空区间。按本题约束，乘入新元素前乘积小于 10^9，乘后小于 10^{15}，long long 足够。时间 $O(n)$，空间 $O(n)$。

完整参考实现 08_window.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; long long k; cin >> n >> k;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    if (k <= 1) { cout << 0 << '\n'; return 0; }
    long long product = 1, answer = 0; int left = 0;
    for (int right = 0; right < n; ++right) {
        product *= a[right];
        while (left <= right && product >= k) product /= a[left++];
        answer += right - left + 1;
    }
    cout << answer << '\n';
}
```

6.6 差值数对与重复元素

给定正整数差 c，若已排序，可对每个值 x 查找 x+c 的出现次数。使用 upper_bound 与 lower_bound 之差可自然处理重复。若 x 有 p 个，x+c 有 q 个，贡献是 pq，而不是 1。例如 $[1,1,3,3,3]$ 且 c=2，答案是 $2 \times 3 = 6$。

当 c=0 时需先确认题目问有序数对还是无序数对，是否允许同一位置使用两次。无序且不同位置的同值组贡献为 $p(p-1)/2$，不能沿用 c>0 的计数公式。

6.7 练习与解答

练习 1。木条 $[1,1]$，k=3，答案是什么？**解答**。0。最大段长哨兵必须覆盖无解情况。

练习 2。窗口题数组 $[1,1,1]$，k=2，答案是多少？**解答**。全部 6 个非空连续区间都合法，分别贡献 1、2、3。k=1 时答案为 0。

练习 3。闭区间最后可行值搜索中，lo=2、hi=3，若使用下中位数且判定为真会怎样？**解答**。mid=2，更新 lo=2 后区间不变，可能死循环。应取上中位数，或采用另一套能证明缩小区间的写法。

7 贪心与基础数学

7.1 贪心需要证明

贪心是在当前状态下确定一个选择并且不回退。它成立需要证明：至少存在一个最优解包含这次选择。常用方法是交换论证，即把某个最优方案换成包含该选择的方案，而目标值不变差。找到几个正确样例不是证明。

例如按价值最大的物品优先装背包并不总是正确：容量 4，物品的（体积，价值）分别为 $(3,5),(2,4),(2,4)$，先拿价值 5 的物品只能得到 5，拿后两件可得 8。基础 01 背包通常需要 DP，不能仅因“选大的”直观就使用贪心。

7.2 例题合并石堆的最小代价

有 n 堆石子，每次可以任意选两堆合并，代价等于两堆石子数之和。求合成一堆的最小总代价。本题**没有相邻限制**。$1 \le n \le 2 \times 10^5$，每堆石子数在 1 到 10^9 之间。输入第一行 n，第二行各堆大小。

输入
4
1 2 3 4
输出
19

先合并 1 和 2，代价 3；再合并 3 和 3，代价 6；最后合并 4 和 6，代价 10。总计 19。

为什么取最小两堆。合并过程可以看成二叉树，每个原始堆是叶子；总代价等于每个叶子权值乘以其深度之和。在固定树形中，较小权值放到较深位置不会使代价增大。选择一对最深的兄弟叶子，可通过交换把最小的两个权值放到那里。因此存在一个最优方案先合并最小两堆；把它们缩成一堆后，剩下同类子问题。重复此选择即可。

用小根堆维护所有堆，每次取两个最小值再插入它们的和。时间 $O(n \log n)$，空间 $O(n)$。只有一堆时无需合并，答案为 0。

完整参考实现 09_merge.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    priority_queue<long long, vector<long long>, greater<long long>> q;
    for (int i = 0; i < n; ++i) { long long x; cin >> x; q.push(x); }
    long long answer = 0;
    while (q.size() > 1) {
        long long x = q.top(); q.pop(); long long y = q.top(); q.pop();
        answer += x + y; q.push(x + y);
    }
    cout << answer << '\n';
}
```

7.3 中位数最小化绝对距离和

若在数轴上选位置 x，使 $\sum_i |a_i - x|$ 最小，排序后取中位数即可。把最左和最右配对：对任何位于两者之间的 x，两项距离和都是两端距离；离开该区间只会增大。对称配对后，所有区间的公共部分包含中位数。偶数个元素时，中间两数之间的任意位置都可达到最小值。

例如 $[1, 2, 9]$ 取 x=2，距离和为 8；取平均数 4，距离和为 10。平均数与中位数解决的目标不同。

7.4 质因数分解与溢出检查

对正整数 x，从 2 开始试除。若 d 整除 x，反复除去并统计指数。循环条件可写 `d <= x/d`，避免计算 d^2 溢出。循环结束后若 x>1，它是最后一个质因子。1 没有质因子。

```
vector<pair<long long,int>> factorize(long long x) {
    vector<pair<long long,int>> answer;
    for (long long d = 2; d <= x/d; ++d) {
        if (x % d != 0) continue;
        int e = 0;
        do { x /= d; ++e; } while (x % d == 0);
        answer.push_back({d,e});
    }
    if (x > 1) answer.push_back({x,1});
    return answer;
}
```

最坏试除次数为 $O(\sqrt{x})$。如果要分解很多较大的数，应重新估计总复杂度，而不是只看一次分解。

7.5 矩阵运算先看维度

若 A 是 $n \times k$，B 是 $k \times m$，则 $C = AB$ 是 $n \times m$，有 $C_{ij} = \sum_{t=1}^{k} A_{it} B_{tj}$，朴素复杂度 $O(nkm)$。矩阵乘法一般不可交换。若题意出现转置或连乘，应先写出各中间矩阵的维度与计算量，再决定结合顺序。计算量可以因结合顺序变化而显著不同，但结果在精确算术下符合结合律。

7.6 练习与解答

练习 1。三堆 $[2, 2, 8]$ 的最优代价是多少？**解答**。先合并 2 和 2 得到 4，代价 4；再与 8 合并代价 12，总计 16。若先合并 2 和 8 则为 22。

练习 2。分解 360。**解答**。$360 = 2^3 \cdot 3^2 \cdot 5$。反复整除才能记录完整指数。

练习 3。A 为 10×100，B 为 100×5，C 为 5×50，先算哪一对更省？**解答**。$(AB)C$ 的乘加规模为 $10 \cdot 100 \cdot 5 + 10 \cdot 5 \cdot 50 = 7500$；$A(BC)$ 为 $100 \cdot 5 \cdot 50 + 10 \cdot 100 \cdot 50 = 75000$，前者更省。

8 搜索与剪枝

8.1 搜索状态和递归职责

写递归前先明确：一次调用负责哪一步、已经作出哪些选择、接下来有哪些合法选择、什么时候记录答案。回溯是撤销本次选择，让下一条分支从同一个父状态开始。常见错误是撤销不完整，导致不同分支互相污染。

8.2 例题按字典序输出全排列

输入 $1 \le n \le 8$，输出 1 到 n 的所有排列，每行一个，按字典序升序。

输入
3
输出
1 2 3
1 3 2
2 1 3
2 3 1
3 1 2
3 2 1

path 保存已选的前缀，used[x] 标记 x 是否已在前缀中。每层从小到大尝试尚未使用的数，输出顺序自然为字典序。每个排列对应唯一一条长度 n 的路径，因此不重不漏。输出本身有 $n \cdot n!$ 个数字量级，时间 $O(n \cdot n!)$，递归与标记空间 $O(n)$。

完整参考实现 10_permutations.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
int n; vector<int> path; vector<bool> used;
void dfs() {
    if ((int)path.size() == n) {
        for (int i = 0; i < n; ++i) cout << path[i] << (i+1 == n ? '\n' : ' ');
        return;
    }
    for (int x = 1; x <= n; ++x) if (!used[x]) {
        used[x] = true; path.push_back(x);
        dfs();
        path.pop_back(); used[x] = false;
    }
}
int main() { cin >> n; used.assign(n+1, false); dfs(); }
```

8.3 剪枝必须有排除理由

剪枝是提前排除不可能产生合法答案或不可能改善答案的分支。可行性剪枝例如皇后问题中已占用的列与对角线；最优性剪枝例如当前代价已不小于已知最优值，且所有后续代价非负。若后续

允许负代价，后一判断就可能失效。

n 皇后按行放置时，记录列 c、对角线 $r+c$、另一方向 $r-c+n$ 是否已使用。每行只放一个皇后，所以行冲突由递归结构自动排除。剪枝减少搜索量，但不意味着最坏复杂度必然变成多项式。

8.4 例题迷宫最短步数

输入 $n \times m$ 字符网格，点号为可走格，井号为障碍。每步上下左右移动一格，求左上角到右下角的最少步数；不可达输出-1。$1 \le n, m \le 500$，起终点也可能是障碍。第一行 n、m，随后 n 行长度为 m 的字符串。

输入
```
3 4
....
.##.
....
```
输出
```
5
```

解答。所有移动代价相等，使用 BFS。起点距离为 0，邻居距离为当前距离加 1；入队时就记录距离，确保同一格最多入队一次。队列按距离层次扩展，因此首次到达就是最短距离。注意这里的“步数”不包含起点，单格可走迷宫答案为 0。时间与空间均为 $O(nm)$。

完整参考实现 11_bfs.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m; cin >> n >> m;
    vector<string> g(n); for (auto &s : g) cin >> s;
    vector<vector<int>> d(n, vector<int>(m, -1));
    queue<pair<int,int>> q;
    if (g[0][0] == '.') { d[0][0] = 0; q.push({0,0}); }
    int dx[] = {-1,1,0,0}, dy[] = {0,0,-1,1};
    while (!q.empty()) {
        auto [x,y] = q.front(); q.pop();
        for (int k = 0; k < 4; ++k) {
            int a = x+dx[k], b = y+dy[k];
            if (a<0 || a>=n || b<0 || b>=m || g[a][b]=='#' || d[a][b]!=-1) continue;
            d[a][b] = d[x][y]+1; q.push({a,b});
        }
    }
    cout << d[n-1][m-1] << '\n';
}
```

8.5 访问标记的三种含义

全排列中的 used 是当前路径内是否使用，返回时必须清除。普通 BFS 的 visited 表示已首次到达，通常全程不清除。记忆化的标记表示这个**完整状态的答案已算出**，应与具体到达路径无关。把

这三种标记混用，会造成漏解、死循环或重复计算。

若走到同一位置时“钥匙是否拿到”影响后续，则状态必须是（位置，钥匙状态）。只用位置作为 visited 会错误合并不同状态。若边权不同，普通 BFS 不再保证最小代价；应根据权值性质考虑 0–1 BFS 或最短路算法，不直接套网格模板。

8.6 练习与解答

练习 1。全排列递归返回后只清 used、不弹出 path 会怎样？**解答**。路径长度与真实递归层数不一致，后续输出混入旧选择，甚至再也无法达到正确终止条件。所有在进入分支时改变的状态都应对应恢复。

练习 2。BFS 为何在入队时标记，而不是出队时？**解答**。同一格可能被多个同层前驱发现，若出队才标记会重复入队。入队时记录距离既防重，也固定首次最短距离。

练习 3。只有两个格子，边代价分别可能为 1 或 100，按经过格子数最少就一定费用最小吗？**解答**。不一定。BFS 最小化边数，只有每条边的代价相等时才等价于最小费用。选择算法前必须看目标函数。

9 动态规划入门

9.1 先定义状态再写转移

动态规划把问题拆成可重复利用的子问题。每道题都要写出五项：状态含义、转移关系、初始值、计算顺序、答案位置。只记转移公式，容易在初值和边界处出错。

所谓无后效性，是状态已经包含决定后续所需的信息。给定状态后，不必再区分到达它的不同历史。这不表示当前选择不会影响未来，当前选择当然会改变后续状态。若过去某项信息会改变可选操作，就应把它加进状态。

例如爬楼梯，每次走 1、2 或 3 阶，令 $f[i]$ 为从 0 阶走到 i 阶的方案数。按最后一步分类，有 $f[i] = f[i-1] + f[i-2] + f[i-3]$。令 $f[0] = 1$，负下标状态为 0。这里的 1 表示“什么也不走”的一种空方案。按 i 从小到大计算，最终答案是 f[n]。

9.2 例题最长严格上升子序列

输入 n 个整数,选择一些位置并保持原有顺序,使选出的值严格递增,求最大长度。$1 \le n \le 5000$，值的绝对值不超过 10^9。子序列可以不连续。第一行 n，第二行数组。

输入

```
6
3 1 2 2 5 4
```

输出

```
3
```

可取 $[1,2,5]$ 或 $[1,2,4]$。设 f[i] 为**以第 i 个元素结尾**的最长长度，

$$f[i] = 1 + \max\{f[j] : j < i,\ a_j < a_i\},$$

没有合法 j 时 f[i]=1。任何以 i 结尾的较长方案，倒数第二个位置必然是一个合法 j；反过来，把 a[i] 接到该 j 的方案后面也合法。因此枚举所有前驱就覆盖全部可能。答案是所有 f[i] 的最大值，不一定是 f[n]。时间 $O(n^2)$，空间 $O(n)$。

完整参考实现 12_lis.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n; vector<int> a(n), f(n,1);
    for (int &x : a) cin >> x;
    int answer = 0;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < i; ++j) if (a[j] < a[i]) f[i] = max(f[i], f[j]+1);
        answer = max(answer, f[i]);
    }
    cout << answer << '\n';
}
```

9.3 例题非空最大子段和

输入 n 个整数，求一个**连续且非空**区间的最大和。$1 \le n \le 2 \times 10^5$，$|a_i| \le 10^9$。

输入

```
5
-2 3 -1 4 -5
```

输出

```
6
```

设 ending 为以当前位置结尾的非空区间最大和。若使用当前数 x，则最优区间要么从 x 重新开始，要么延长前一位置的最优区间：$ending = \max(x, ending + x)$。全局答案记录所有位置的最大 ending。由于要求非空，初始化为第一个元素，不能初始化为 0。时间 $O(n)$，额外空间 $O(1)$。

完整参考实现 13_maxsum.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n; long long x; cin >> x;
    long long ending = x, answer = x;
    for (int i = 1; i < n; ++i) {
        cin >> x; ending = max(x, ending+x); answer = max(answer, ending);
    }
    cout << answer << '\n';
}
```

9.4 例题数字三角形的最大路径和

输入 n 行数字，第 i 行有 i 个数。从顶点出发，每次走到下一行的正下方或右下方，求到最后一行的最大路径和。$1 \le n \le 1000$，每个数的绝对值不超过 10^6。第一行 n，随后各行给出三角形。

输入
3
2
-1 3
4 -2 1
输出
6

二维状态 f[i][j] 表示到第 i 行第 j 个位置的最优值，合法前驱只有 $(i-1,j-1)$ 与 $(i-1,j)$。初值为顶点，非法位置为负无穷。转移

$$f[i][j] = a[i][j] + \max(f[i-1][j-1], f[i-1][j]).$$

上例路径 $2,3,1$ 得 6。因为每行只依赖前一行，可压缩成一维。**从右向左更新**才能确保 f[j-1] 尚未被本行覆盖。代码设置 f[1]=0 作为顶点前的虚拟起点，其他位置为负无穷。时间 $O(n^2)$，额外空间 $O(n)$。

完整参考实现 14_triangle.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    const long long NEG = -(1LL<<60);
    vector<long long> f(n+1, NEG); f[1] = 0;
    for (int i = 1; i <= n; ++i) {
        vector<long long> row(i+1);
        for (int j = 1; j <= i; ++j) cin >> row[j];
        for (int j = i; j >= 1; --j) f[j] = max(f[j], f[j-1]) + row[j];
    }
    cout << *max_element(f.begin()+1, f.end()) << '\n';
}
```

9.5 二维 DP 与最长公共子序列

给两个长度分别为 n、m 的序列，令 $f[i][j]$ 为两个前缀的最长公共子序列长度。若最后两个元素相等，候选为 $f[i-1][j-1]+1$；还可舍弃任意一边末元素。统一写为

$$f[i][j] = \max(f[i-1][j], f[i][j-1],\ [a_i = b_j] \text{ 时的} f[i-1][j-1]+1).$$

空前缀长度为 0，按行列递增计算。时间 $O(nm)$，滚动数组可把空间压缩到 $O(m)$，但仍不减少时间。若 n、m 都到 10^5，仅压缩空间仍不能通过，需要利用题目的额外性质。

9.6 记忆化搜索

记忆化按需求递归计算状态，求完后缓存。它与按顺序递推的主要区别是访问顺序。缓存键必须包含完整状态；若结果还取决于一个没有放进键里的参数，缓存会错误复用。

不要简单地以“结果为 0”判断是否计算过，因为 0 可能是合法答案。可以单设 done 数组。一般 DP 要求依赖关系可以按某种顺序求值；有环图上的最短路不能靠“加个缓存”就自动解决。第 11 章模板展开会同时使用“正在计算”和“已完成”两种标记识别环。

9.7 练习与解答

练习 1。数组 $[5,4,3]$ 的严格 LIS 长度是多少？**解答**。1。每个位置的 f 都为 1；若题目要求不下降序列，应把比较条件改为不大于，而不是仍用严格小于。

练习 2。数组 $[-5,-2,-7]$ 的非空最大子段和是多少？**解答**。-2。初始化为 0 会错误地选择空区间。

练习 3。三阶楼梯、每步走 1、2 或 3 阶，共多少方案？**解答**。4 种：111、12、21、3。由 $f[0]=1, f[1]=1, f[2]=2$ 得 $f[3]=4$。

练习 4。滚动数组能否把 LCS 的 $O(nm)$ 时间降为 $O(m)$？**解答**。不能。它只减少保存的历史行数，仍需要计算 n 行各 m 个状态。

10 基础背包与计数

10.1 先确定问题问的是什么

背包的“容量”可以是时间、预算或其他资源，“价值”可以是收益、数量或得分。建模前必须区分：资源是最多使用还是恰好使用；每件只能选一次还是可无限选择；目标是最大价值、最小代价、能否达到还是方案数量。

状态含义	初始化	典型答案
容量不超过 v 的最大价值	全 0，允许什么都不选	f[V]。
恰好用 v 的最大价值	f[0]=0，其余负无穷	f[V]；不可达需另行输出。
恰好用 v 的最小代价	f[0]=0，其余正无穷	f[V]。
恰好凑出 v 的方案数	f[0]=1，其余 0	f[V]，必要时取模。

10.2 例题每件物品最多选一次

有 n 件物品，每件体积 c、价值 w，容量为 V，求总体积不超过 V 的最大总价值。$1 \le n \le 200$，$0 \le V \le 5000$，$1 \le c \le 5000$，$0 \le w \le 10^9$。输入第一行 n、V，后面每行 c、w。

输入
3 5
2 3
3 4
4 5
输出
7

令 f[i][v] 为只考虑前 i 件物品、容量不超过 v 的最大价值。第 i 件选或不选：

$$f[i][v] = \max(f[i-1][v],\ f[i-1][v-c_i] + w_i),$$

第二项只在 $v \geq c_i$ 时存在。若压缩掉 i 维，容量必须从大到小更新，使 f[v-c] 仍是上一层状态，避免同一件被重复选。复杂度 $O(nV)$，空间 $O(V)$。

完整参考实现 15_knapsack01.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, capacity; cin >> n >> capacity;
    vector<long long> f(capacity+1);
    for (int i = 0; i < n; ++i) {
        int cost; long long value; cin >> cost >> value;
        for (int v = capacity; v >= cost; --v) f[v] = max(f[v], f[v-cost]+value);
    }
    cout << f[capacity] << '\n';
}
```

10.3 例题每种物品可以无限选择

输入格式与上一题相同，但现在 n 表示物品种类，每种可以选任意非负整数件。仍要求体积不超过 V、最大化价值，且体积必须为正。采用同样的约束。

输入
2 7
2 3
3 4
输出
10

选择体积 2 的两件和体积 3 的一件，价值 10。二维转移的选取项变为 $f[i][v-c_i]+w_i$，它允许当前种类继续使用。因此一维容量循环从小到大，使 f[v-c] 已经包含本种物品。时间 $O(nV)$，空间 $O(V)$。

完整参考实现 16_knapsack_complete.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, capacity; cin >> n >> capacity;
    vector<long long> f(capacity+1);
    for (int i = 0; i < n; ++i) {
        int cost; long long value; cin >> cost >> value;
        for (int v = cost; v <= capacity; ++v) f[v] = max(f[v], f[v-cost]+value);
    }
    cout << f[capacity] << '\n';
}
```

10.4 用最小反例检查循环方向

只有一件体积 2、价值 3 的物品，容量 4。01 背包答案为 3；若正序更新，计算 f[4] 时使用了本轮刚更新的 f[2]=3，得到 6，实际上重复选择了同一件。完全背包允许重复，答案才是 6。记住“逆序 01、正序完全”之外，更要能解释读到的是哪一层。

10.5 例题无限硬币的组合数

有 n 种**互不相同的正整数面额**，每种无限个，问恰好凑出 target 的组合数，硬币选择顺序不算不同方案。输出对 10^9+7 取模的结果。$1 \le n \le 100$，$0 \le target \le 10^4$，面额不超过 10^4。输入第一行 n、target，第二行所有面额。

输入
3 5
1 2 5
输出
4

四种为 $1+1+1+1+1$、$1+1+1+2$、$1+2+2$、5。令 f[v] 为已处理面额能凑出的组合数。先枚举面额，再正序枚举金额，确保每种组合按固定面额顺序生成一次。f[0]=1 是构造非空方案的起点。时间 $O(n \cdot target)$，空间 $O(target)$。

完整参考实现 17_coin_count.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, target; cin >> n >> target;
    const int MOD = 1000000007;
    vector<int> f(target+1); f[0] = 1;
    for (int i = 0; i < n; ++i) {
        int c; cin >> c;
        for (int v = c; v <= target; ++v) f[v] = (f[v]+f[v-c])%MOD;
    }
    cout << f[target] << '\n';
}
```

若改成先枚举金额、再枚举最后一个硬币，就会按排列计数。例如面额 1 和 2 凑 3，组合只有 $1+1+1$ 与 $1+2$ 两种，排列则还区分 $1+2$ 和 $2+1$，共三种。循环顺序改变了状态的计数含义。

10.6 例题每组最多选一件

有 g 组物品,每组最多选一件,也可以不选。求容量 V 内的最大价值。$1 \le g \le 100, 0 \le V \le 2000$，每组 1 到 20 件，体积为正、价值非负。输入第一行 g、V，随后每组先给物品数 k，再给 k 对体积、价值。

输入

```
2 5
2
2 3
3 5
2
2 4
4 7
```

输出

```
9
```

第一组选体积 3 价值 5，第二组选体积 2 价值 4。对每组保留旧数组 f，所有选择都从旧状态转移到 next，next 初始复制 f 表示不选。这种写法直接保证同一组至多贡献一件，比混用循环顺序更容易检查。若总物品数为 M，时间 $O(MV)$，空间 $O(V)$。

完整参考实现 18_group.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int groups, capacity; cin >> groups >> capacity;
    vector<long long> f(capacity+1);
    for (int g = 0; g < groups; ++g) {
        int k; cin >> k; vector<pair<int,long long>> items(k);
        for (auto &[c,w] : items) cin >> c >> w;
        auto next = f;
        for (int v = 0; v <= capacity; ++v)
            for (auto [c,w] : items) if (v >= c) next[v] = max(next[v], f[v-c]+w);
        f.swap(next);
    }
    cout << f[capacity] << '\n';
}
```

10.7 恰好装满时如何处理不可达

将所有状态设为负无穷，仅 f[0]=0。在转移前检查前驱可达，再进行加法。代码可使用约 -4×10^{18} 的哨兵，但必须证明合法值不会接近它；也可用额外布尔数组表示可达。不能把不可达初始化为 0，否则相当于凭空允许每个体积都有一个合法空方案。

10.8 练习与解答

练习 1。只有体积 2 价值 3 的一件物品，容量 3 且要求恰好装满，答案是什么？**解答**。不可达；若要求不超过容量，答案为 3。两种问法的初始化和答案解释不同。

练习 2。面额 2、3 凑 0 的组合数是多少？**解答**。1，即不取任何硬币。这也是 f[0]=1 的组合含义。

练习 3。只有一组，包含体积 2 价值 3、体积 3 价值 5，容量 5。答案是多少？**解答**。5，不能同组选两件得到 8。使用旧数组转移可防止这个错误。

练习 4。若完全背包出现体积 0 且价值正的物品，答案还有限吗？**解答**。没有有限上界。本章题目明确要求体积为正，这是算法与问题有意义的重要前提。

11 第三题综合模拟专题

11.1 长题意的实现流程

先写一张规则表：输入是什么、状态存在哪里、操作改变什么、查询输出什么、异常如何处理。再按职责拆成解析层、状态层和执行层。只有确认每层的输入输出后，再连成完整程序。下面五个案例规则完整，分别训练路径、映射状态、递归解析、树形结构和依赖展开，可以脱离外部题面学习。

11.2 例题路径规范化

题目。给定固定的当前目录 cwd 和 q 条路径。以斜杠开头的是绝对路径，否则相对于 cwd。连续斜杠视为一个；点号目录表示当前目录；两个点号表示父目录，在根目录继续上移仍为根。普通目录名只含字母、数字、下划线。路径不含空格且非空。

输出规范绝对路径：以一个斜杠开头，不含点号、两个点号或重复斜杠；除根目录外不以斜杠结尾。**每条查询独立使用原来的 cwd，不改变当前目录**。路径长度不超过 10^4，全部输入路径总长度不超过 2×10^5，$1 \le q \le 10^4$。第一行 cwd，第二行 q，随后 q 行各一条路径。cwd 保证为绝对路径，也允许特殊片段。

```
输入
/a/b
5
../c
/x//./y/../
../../../
.
tmp/../../z
输出
/a/c
/x
/
/a/b
/a/z
```

解答。用 vector 保存从根向下的目录名。普通片段入栈，两个点号在非空时弹栈，空片段和点号忽略。绝对路径从空栈开始，相对路径从 cwd 的副本开始。

最后一条查询从 $[a, b]$ 出发，读 tmp 得到 $[a, b, tmp]$，两次父目录操作后为 $[a]$，加入 z 后得到 $[a, z]$。不变量是：栈中的目录始终表示已经处理的路径前缀的规范形式。

设 cwd 长度为 B，第 i 个查询长度为 L_i。复制基目录的总成本最坏为 $O(qB)$，加上扫描与输出为 $O(qB + \sum L_i)$。不能漏算复制和输出中重复的 cwd 长度。

完整参考实现 19_path.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
void append_parts(const string &s, vector<string> &parts) {
    string item; stringstream input(s);
    while (getline(input, item, '/')) {
        if (item.empty() || item == ".") continue;
        if (item == "..") { if (!parts.empty()) parts.pop_back(); }
        else parts.push_back(item);
    }
}
int main() {
    string cwd; cin >> cwd; int q; cin >> q;
    vector<string> base; append_parts(cwd, base);
    while (q--) {
        string path; cin >> path;
        vector<string> result = (path[0]=='/' ? vector<string>{} : base);
        append_parts(path, result);
        if (result.empty()) cout << '/';
        for (const string &x : result) cout << '/' << x;
        cout << '\n';
    }
}
```

调试。先测根目录与普通绝对路径，再测相对路径和多次上移，最后检查是否错误地修改了共享 cwd。若后续查询受前面查询影响，优先检查是否复用了同一可变目录数组。

11.3 例题授权和撤销的状态管理

题目。执行 q 条命令：`GRANT user resource` 授予权限；`REVOKE user resource` 撤销；`CHECK user resource` 查询并输出 YES 或 NO。重复授予不叠加；撤销不存在的权限不报错；未出现用户默认无权限。操作依次执行，只有 CHECK 输出。$1 \le q \le 2 \times 10^5$，名称为 1 到 20 个字母或数字。本题不含角色继承、通配符或优先级。

输入

```
8
CHECK alice file1
GRANT alice file1
GRANT alice file1
CHECK alice file1
REVOKE alice file1
CHECK alice file1
REVOKE bob file2
CHECK bob file2
```

输出

```
NO
YES
NO
NO
```

解答。外层 map 按用户名定位，内层 set 保存资源。set 处理重复授权。查询和撤销用 find，避免无意创建空记录。所有操作依次执行，因此 CHECK 反映此前的状态。

设用户数 U，每位用户最多持有 R 项权限，单次比较次数为 $O(\log U + \log R)$，还需考虑字符串比较长度。总存储不超过授予过的记录规模。权限是用户与资源的二元关系，不能只判断用户是否出现过；撤销 file1 不能影响 file2。

完整参考实现 20_permissions.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int q; cin >> q;
    map<string, set<string>> rights;
    while (q--) {
        string op, user, resource; cin >> op >> user >> resource;
        if (op == "GRANT") rights[user].insert(resource);
        else if (op == "REVOKE") {
            auto it = rights.find(user);
            if (it != rights.end()) it->second.erase(resource);
        } else {
            auto it = rights.find(user);
            bool allowed = it != rights.end() && it->second.count(resource);
            cout << (allowed ? "YES" : "NO") << '\n';
        }
    }
}
```

11.4 例题带括号的整数表达式

题目。输入一行表达式，支持非负整数字面量、加、减、乘、括号和一元正负号。乘法优先于加减，同级二元运算从左到右；一元符号优先于乘法；只忽略普通空格。合法时输出结果，语法错误输出 ERROR。不支持除法、变量和隐式乘法，例如 `2(3+4)` 非法。

行长不超过 1000，括号与连续一元符号嵌套不超过 100 层，字面量不超过 10^6；保证求值过程中涉及的数值运算的中间结果绝对值不超过 10^{12}，本题只讨论语法错误，不要求处理溢出。

输入

```
2 * (3 + 4) - -5
```

输出

```
19
```

另一组输入

```
1 + (2 * 3
```

另一组输出

```
ERROR
```

按语法拆函数。

```
expression = term { ('+' | '-') term }
term       = factor { '*' factor }
factor     = number | '(' expression ')' | '+' factor | '-' factor
```

expression 负责加减，term 负责乘法，factor 负责括号、数字与一元符号。共享指针 p 指向下一个未处理字符，每个函数返回其负责部分的值。term 先完整处理乘法，从而保证优先级；循环保证减法左结合，例如 $8-3-2=3$。

解析前面一个合法表达式不代表整行合法。例如 `1 2`，解析 1 后还剩 2，必须输出 ERROR。最后检查是否读到行尾。时间 $O(L)$，递归空间与嵌套深度有关。

完整参考实现 21_expression.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
struct Parser {
    string s; size_t p = 0;
    void space() { while (p<s.size() && s[p]==' ') ++p; }
    bool take(char c) {
        space(); if (p<s.size() && s[p]==c) { ++p; return true; }
        return false;
    }
    long long expression() {
        long long x = term();
        while (true) {
            if (take('+')) x += term();
            else if (take('-')) x -= term();
            else return x;
        }
    }
    long long term() {
        long long x = factor();
        while (take('*')) x *= factor();
        return x;
    }
    long long factor() {
        if (take('+')) return factor();
        if (take('-')) return -factor();
        if (take('(')) {
            long long x = expression();
            if (!take(')')) throw runtime_error("missing )");
            return x;
        }
        space();
        if (p==s.size() || !isdigit((unsigned char)s[p])) throw runtime_error("number expected");
        long long x = 0;
        while (p<s.size() && isdigit((unsigned char)s[p])) x = x*10+s[p++]-'0';
        return x;
    }
};
int main() {
    Parser a; getline(cin, a.s);
    try {
        long long answer = a.expression(); a.space();
        if (a.p != a.s.size()) throw runtime_error("trailing input");
        cout << answer << '\n';
    } catch (const exception &) { cout << "ERROR\n"; }
}
```

调试。依次验证单数字、一元负号、连续加减、乘法优先级、嵌套括号，再测空串、缺右括号、多右括号和多余字符。查看每个函数消费到哪里，比只看最终错误数值更有帮助。

11.5 例题树形文档的后代选择查询

题目。给出按先序排列的一片有根森林，共 n 个节点。每行记录深度、标签和标识。根深度 0，孩子比父亲深 1；输入结构合法，每次深度最多增加 1。标识为减号表示无标识，其余标识互不相同；标签和标识只含字母数字。

查询由空格分隔的选择符组成。普通选择符匹配标签，以井号开头的选择符匹配标识。`div p` 表示匹配 p 节点，且它的某个**严格祖先**是 div；`div div p` 需要两个不同的 div 祖先按顺序出现，允许隔着其他节点。节点不能把自己当祖先。

$1 \le n \le 500$，$1 \le q \le 50$，最大深度 100，每条查询 1 到 10 个选择符。第一行 n、q，随后 n 行节点，最后 q 行查询。每次输出匹配数和按输入顺序排列的节点行号，编号从 1 开始。

```
输入
7 5
0 html -
1 body -
2 div main
3 p one
2 div side
3 p two
2 p three
p
div p
#main p
div div p
#side
输出
3 4 6 7
2 4 6
1 4
0
1 5
```

建树。用祖先栈。读取深度 d 的节点时，把栈截断到长度 d；栈顶就是父亲，空栈则为根，然后压入自己。读取同深度兄弟时，旧兄弟会被弹出。

匹配。从查询末尾开始，先检查候选节点本身，再沿父链逆序匹配剩余选择符。找到一个就继续寻找更高祖先。选择最近的匹配祖先不会损失可行解，因为它给剩余前缀保留最多的上方祖先。找齐全部选择符则接受。

候选检查最多走树高 H 条父边并处理 K 个选择符，整体为 $O(qn(H+K))$，本题规模下足够。节点 7 虽是 p，但没有 div 祖先；若程序把它算入 `div p`，通常是把 “之前出现的节点” 误认为 “当前祖先”。

完整参考实现 22_selector.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
struct Node { string tag, id; int parent; };
int main() {
    int n, q; cin >> n >> q;
    vector<Node> a(n); vector<int> ancestors;
    for (int i = 0; i < n; ++i) {
        int depth; cin >> depth >> a[i].tag >> a[i].id;
        while ((int)ancestors.size() > depth) ancestors.pop_back();
        a[i].parent = ancestors.empty() ? -1 : ancestors.back();
        ancestors.push_back(i);
    }
    string line; getline(cin, line);
    auto matches = [&](int u, const string &token) {
        if (token[0]=='#') return a[u].id != "-" && a[u].id == token.substr(1);
        return a[u].tag == token;
    };
    while (q--) {
        getline(cin, line); stringstream ss(line); vector<string> tokens;
        string token; while (ss >> token) tokens.push_back(token);
        vector<int> answer;
        for (int u = 0; u < n; ++u) {
            int k = (int)tokens.size()-1;
            if (!matches(u, tokens[k])) continue;
            --k; int v = a[u].parent;
            while (v != -1 && k >= 0) {
                if (matches(v, tokens[k])) --k;
                v = a[v].parent;
            }
            if (k < 0) answer.push_back(u+1);
        }
        cout << answer.size(); for (int x : answer) cout << ' ' << x; cout << '\n';
    }
}
```

11.6 例题模板展开与循环依赖

题目。有 n 个命名模板，每个由若干以空白分隔的词组成。以 at 符号开头的词引用另一个模板，其余词直接输出。定义全部读完后，展开指定起始模板，以单个空格连接所有普通词。从起始模板可达的引用若出现未定义名称或环，输出 ERROR；不可达模板的错误不影响本次输出。

模板名互不重复，只含字母数字；引用符号后必须有名称。允许空模板。$1 \le n \le 100$，每个定义最多 100 个词，每词长度最多 20。保证无环情况下每个模板完整展开不超过 2000 个词。第一行 n，随后每个定义包含名称、词数 k 及 k 个词，最后给出起始名称。

输入

```
3
title 1 world
line 3 hello @title !
all 2 @line @line
all
```

输出

```
hello world ! hello world !
```

另一组输入

```
2
a 1 @b
b 1 @a
a
```

另一组输出

```
ERROR
```

三色状态。0 表示未算，1 表示正在递归，2 表示已经完成并缓存。访问状态 1 说明当前依赖链回到了自己；访问状态 2 直接复用。第二次引用同一模板不必然成环，关键看第一次展开是否结束。

解答。先读入全部定义以允许前向引用。按词序展开，普通词加入结果，引用词接上被引用模板的结果。记忆化避免重算，但每次拼接仍复制相应词。时间取决于查找及所有实际复制的词数；本题每个模板最多缓存 2000 词，总缓存约不超过 100×2000 词，不能忽略输出规模而声称总是 $O(n)$。

完整参考实现 23_templates.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
map<string, vector<string>> definitions, memo;
map<string, int> state;
const vector<string> &expand(const string &name) {
    if (!definitions.count(name)) throw runtime_error("unknown name");
    if (state[name] == 1) throw runtime_error("cycle");
    if (state[name] == 2) return memo.at(name);
    state[name] = 1;
    vector<string> output;
    for (const string &token : definitions.at(name)) {
        if (token[0] != '@') output.push_back(token);
        else {
            const auto &part = expand(token.substr(1));
            output.insert(output.end(), part.begin(), part.end());
        }
    }
    state[name] = 2; memo[name] = move(output); return memo.at(name);
}
int main() {
    int n; cin >> n;
    while (n--) {
        string name; int k; cin >> name >> k;
        auto &tokens = definitions[name]; tokens.resize(k);
        for (auto &s : tokens) cin >> s;
    }
    string start; cin >> start;
    try {
        const auto &out = expand(start);
        for (size_t i = 0; i < out.size(); ++i) cout << (i ? " " : "") << out[i];
        cout << '\n';
    } catch (const exception &) { cout << "ERROR\n"; }
}
```

11.7 练习与解答

练习 1。cwd 为根，路径为 `../../a`，结果是什么？**解答**。`/a`，超出根的上移被忽略。

练习 2。alice 有 A 和 B，撤销 A 后查询 B 输出什么？**解答**。YES，资源权限分别维护。

练习 3。`-2*-3+4` 的值是多少？**解答**。10。一元负号属于 factor，乘法先得 6，再加 4。

练习 4。一个 div 节点能否独自匹配 `div div`？**解答**。不能，两个选择符必须匹配不同节点，前一个位于严格祖先链。

练习 5。A 引用 B 两次，B 只含 x，是否有环？**解答**。没有。第一次结束后 B 为状态 2，第二次复用，输出 `x x`。

12　综合训练与复盘

12.1　练习方式

本章三道题均为教学训练题，不对应某次认证的原版试卷。建议先不看解答，给自己安排 30、50、90 分钟的独立尝试，再用解答检查建模过程。用时只是训练建议，不是官方考试分配。

12.2　例题训练 A 差值数对

给 n 个整数和非负整数 c，求满足 $i < j$ 且 $|a_i - a_j| = c$ 的位置对数量。$1 \le n \le 2 \times 10^5$，$|a_i| \le 10^9$，$0 \le c \le 2 \times 10^9$。第一行 n、c，第二行数组，输出一个整数。

输入
5 2
1 3 1 3 3
输出
6

解答。先统计每个值的频次。c>0 时，只从较小值 x 寻找 x+c，不会重复统计同一无序数对；贡献是两组频次乘积。c=0 时，频次 k 的组中任选两个位置，贡献 $k(k-1)/2$。答案最多为 $n(n-1)/2$，需要 long long。时间 $O(n \log n)$，空间 $O(n)$。

完整参考实现　24_pairs.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; long long c; cin >> n >> c;
    map<long long,long long> count;
    for (int i = 0; i < n; ++i) { long long x; cin >> x; ++count[x]; }
    long long answer = 0;
    for (auto [x, k] : count) {
        if (c == 0) answer += k*(k-1)/2;
        else { auto it = count.find(x+c); if (it != count.end()) answer += k*it->second; }
    }
    cout << answer << '\n';
}
```

检查用例。数组 [7, 7, 7]、c=0 应输出 3；单元素时应输出 0。若输出 9，说明把相同位置配对或对称重复都算了进去。

12.3　例题训练 B 矩形增量的最大值

初始有 $n \times m$ 全 0 矩阵，执行 q 次操作，每次给闭矩形左上 (x_1, y_1) 到右下 (x_2, y_2) 加 v。最终输出最大元素值，以及等于最大值的格子数。$1 \le n, m \le 1000$，$0 \le q \le 2 \times 10^5$，$|v| \le 10^6$，坐标合法。第一行 n、m、q，随后每行五个数。

输入

```
3 3 2
1 1 2 2 2
2 2 3 3 3
```

输出

```
5 1
```

解答。把每次矩形更新记录在二维差分四角，最后恢复每格真实值，同时维护最大值和次数。只统计 1 到 n 行、1 到 m 列，不能把辅助边界算进去。允许负更新，最大值要从负无穷开始，不能默认 0。时间 $O(nm+q)$，空间 $O(nm)$。

完整参考实现 25_rectangle.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, m, q; cin >> n >> m >> q;
    vector<vector<long long>> d(n+2, vector<long long>(m+2));
    while (q--) {
        int x1,y1,x2,y2; long long v; cin >> x1 >> y1 >> x2 >> y2 >> v;
        d[x1][y1] += v; d[x2+1][y1] -= v;
        d[x1][y2+1] -= v; d[x2+1][y2+1] += v;
    }
    long long best = LLONG_MIN, count = 0;
    for (int i = 1; i <= n; ++i) for (int j = 1; j <= m; ++j) {
        d[i][j] += d[i-1][j] + d[i][j-1] - d[i-1][j-1];
        if (d[i][j] > best) { best = d[i][j]; count = 1; }
        else if (d[i][j] == best) ++count;
    }
    cout << best << ' ' << count << '\n';
}
```

检查用例。q=0 时输出 `0 n*m`，第二项应计算成实际格子数；若整个矩阵一次减 5，则最大值为-5 而不是 0。

12.4 例题训练 C 非抢占任务调度

有一台处理器和 n 个任务，每个任务给定到达时刻、优先级和执行时间，输入顺序就是任务编号。处理器空闲时，在所有已经到达但尚未执行的任务中选择优先级最高者；并列时选到达更早者，再并列选编号更小者。一旦开始执行就**不能被抢占**。

一个任务完成时，必须先把恰好在该时刻到达的任务加入候选，再选择下一项。若没有已到达任务，时钟直接跳到下一次到达。输入不保证按到达时刻有序。$1 \le n \le 2 \times 10^5$，到达时刻和优先级在 0 到 10^9 之间，执行时间在 1 到 10^9 之间。

第一行 n，随后每行依次给到达时刻、优先级、执行时间。按输入编号输出每个任务的开始和结束时刻，最后一行输出最大等待时间，等待时间为开始减到达。

输入

```
4
0 1 5
1 1 2
2 3 1
5 2 2
```

输出

```
0 5
8 10
5 6
6 8
7
```

推演。任务 1 在 0 到 5 执行，中途到达的高优先级任务不能打断它。时刻 5，先加入任务 4；待选任务 2、3、4 中任务 3 优先级最高，所以执行顺序为 1、3、4、2。任务 2 等到 8 开始，等待 7。

状态划分。排序数组保存还未到达的任务，指针 i 指向下一项；优先队列保存已经到达的任务；now 保存处理器可再次分配工作的时刻。每轮先补全所有到达时间不大于 now 的任务，再选堆顶。

正确性。选择前，堆恰好包含全部可选任务；比较器严格实现三级优先顺序。执行结束后更新 now，不处理中途抢占，符合规则。每个任务入堆出堆各一次，排序与堆操作总时间 $O(n \log n)$，空间 $O(n)$。结束时刻可能达到 10^{14} 量级，应使用 long long。

完整参考实现 26_scheduler.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
struct Task { long long arrival, priority, duration; int id; };
struct Compare {
    bool operator()(const Task &a, const Task &b) const {
        if (a.priority != b.priority) return a.priority < b.priority;
        if (a.arrival != b.arrival) return a.arrival > b.arrival;
        return a.id > b.id;
    }
};
int main() {
    int n; cin >> n; vector<Task> tasks(n);
    for (int i = 0; i < n; ++i) {
        cin >> tasks[i].arrival >> tasks[i].priority >> tasks[i].duration;
        tasks[i].id = i;
    }
    sort(tasks.begin(), tasks.end(), [](auto a, auto b) {
        return tie(a.arrival,a.id) < tie(b.arrival,b.id);
    });
    priority_queue<Task,vector<Task>,Compare> ready;
    vector<pair<long long,long long>> answer(n);
    long long now = 0, maxWait = 0; int i = 0;
    while (i < n || !ready.empty()) {
        if (ready.empty()) now = max(now, tasks[i].arrival);
        while (i < n && tasks[i].arrival <= now) ready.push(tasks[i++]);
        Task t = ready.top(); ready.pop();
        answer[t.id] = {now, now+t.duration};
        maxWait = max(maxWait, now-t.arrival); now += t.duration;
    }
    for (auto [start,finish] : answer) cout << start << ' ' << finish << '\n';
    cout << maxWait << '\n';
}
```

12.5 如何复盘一次错误

先保存最小失败输入，写出期望输出，再归类原因。输入解析错误要检查分词和换行；状态错误要明确变量对应时刻；算法错误要重新验证单调性、转移或贪心理由；复杂度错误要估计最坏规模。修复后还要补一个能够稳定复现原错误的测试。

例如调度程序把新任务加入堆的条件写成“到达时刻小于 now”，就会漏掉恰好在任务结束时到达的任务。用“任务 1 在 0 开始、执行 2；任务 2 在 2 到达”即可定位边界。改成小于等于后，再检查已有等待任务与同时到达任务的优先顺序。

12.6 提交前检查表

1. 输出内容、顺序和格式是否与题意一致，有没有调试输出残留。
2. 最小输入、全相等、全负数、空结果、不可达等合法边界是否处理。

3. 数值和数组范围是否足够，r+1、i-1 等辅助位置是否分配。
4. 每个循环能否终止；每个递归状态是否完整；是否可能爆栈。
5. 每个容器访问前是否检查为空或 end；比较器是否严格。
6. 程序复杂度是否覆盖最大数据，而不仅是样例。

A 提高专题与模型扩展

本附录承接原资料中的提高内容。建议在完成正文例题后选择学习。每次变换模型时，先检查状态语义是否改变，不要仅凭代码形式相似就替换循环。

A.1 多重背包与二进制拆分

第 i 种物品最多有 s_i 件。朴素转移枚举件数 k：

$$f[i][v] = \max_{0\le k\le s_i,\ kc_i\le v}\{f[i-1][v-kc_i]+kw_i\}.$$

当数量较大时，把 s_i 件拆成若干只能选一次的物品包。例如 13 拆成 1、2、4、6，每包体积和价值乘相应件数。选择这些包可以表示 0 到 13 的任意件数，转化为 01 背包。拆包数量为 $O(\log(s_i+1))$，总时间为 $O(V\sum_i \log(s_i+1))$。

```
int remaining = count;
for (int batch = 1; remaining > 0; batch *= 2) {
    int take = min(batch, remaining);
    remaining -= take;
    long long cc = 1LL * take * cost;
    long long ww = 1LL * take * value;
    if (cc <= capacity)
        for (int v = capacity; v >= cc; --v)
            f[v] = max(f[v], f[v-(int)cc] + ww);
}
```

此片段假设 count 不超过 10^6、体积为正，避免 batch 翻倍溢出。若采用更大数量范围，应使用更宽类型并检查翻倍边界。

为什么能覆盖。若已有包可以组成 0 到 t，再加入大小不超过 t+1 的新包，就可以连续覆盖到 t 加新包大小。按 1、2、4 增长并保留余数满足这个条件。

计数陷阱。这个拆分通常用于最优值或可达性，不能直接拿来统计原问题的方案数。例如拆成 1、2、4、6 后，6 可由 6 或 2+4 表示，形成不同拆包选择却代表同样件数。计数背包应另行设计不重不漏的转移。

A.2 混合背包与多维费用

混合背包对每种物品分别判断：限一次用逆序，无限次用正序，有限次可枚举或拆包；组内互斥则按整组处理，不能把同组物品当普通独立物品。不同模型混在一起时，应确保每轮转移的“一个决策单位”仍然清晰。

若每件物品同时占体积 c 和重量 d，容量限制为 V 和 W，定义 $f[v][w]$ 为两种资源限制下的最大价值。对于 01 物品，两个容量维度都逆序：

```
for (int v = V; v >= cost; --v)
    for (int w = W; w >= weight; --w)
        f[v][w] = max(f[v][w], f[v-cost][w-weight]+value);
```

时间 $O(nVW)$，空间 $O(VW)$。增加状态维度会迅速增加计算量。应选择取值范围较小的量作维度，必要时考虑按价值记录最小成本，但也要重新定义答案和不可达值。

A.3 方案恢复

若要求输出选了哪些物品，仅得到最大值还不够。对于二维 01 背包，可在每个状态记录是否选择第 i 件，或保留整张 f 表，从 (n, V) 逆推：若最优值来自不选则 i 减 1；若来自选择，则记录 i 并把容量减去体积。

一维压缩后直接保存“当前容量的父指针”容易被后续物品覆盖，造成恢复结果重复选同一物品。为了初学者的可靠实现，先使用二维记录恢复，再考虑持久化等高级优化。并列最优解允许任选时，固定一种规则即可；要求字典序最小时必须把并列规则也纳入设计。

A.4 计数背包的撤销

设 F[s] 是所有物品组成体积 s 的 01 选择方案数，删除一个体积为 w 的**指定物品**后，G[s] 是剩余方案数。按是否选该物品分类：

$$F[s] = G[s] + G[s-w], \qquad G[s] = F[s] - G[s-w].$$

负下标项为 0。因 G[s] 依赖更小的 G，按 s 从小到大计算。若只输出末位数字，应在模 10 下减法并把负数归一化：

```
vector<int> g = all;
for (int s = w; s <= capacity; ++s)
    g[s] = (all[s] - g[s-w] + 10) % 10;
```

例如两个体积都为 1 的不同物品，完整方案数为 $F[0..2] = [1, 2, 1]$；删去其中一件，得到 $G = [1, 1, 0]$。物品即使体积相同也按身份区分。该线性计数关系可撤销，但最大值背包的 max 会丢失候选信息，不能简单相减恢复。

A.5 LIS 优化与排列 LCS

定义 tail[len] 为所有长度 len 的严格上升子序列中最小的末尾值。这个数组严格递增。处理 x 时，用 lower_bound 找到第一个不小于 x 的位置并替换；若不存在就把 x 接到末尾。替换更小的末尾不会改变已获得的长度，却给后续元素留下更多延伸可能。

```
vector<int> tail;
for (int x : a) {
    auto it = lower_bound(tail.begin(), tail.end(), x);
    if (it == tail.end()) tail.push_back(x);
    else *it = x;
}
cout << tail.size();
```

时间 $O(n \log n)$。tail 的内容不一定是一条真实的 LIS；若要求恢复路径，还需要记录位置和前驱。不下降子序列对应 upper_bound，而不是 lower_bound。

若两个序列都是 1 到 n 的排列，把第一序列每个值的位置记为 pos，再将第二序列映射成位置序列。共同子序列需要在第一序列位置上递增，因此转成 LIS。这个转换利用了元素唯一的性质，不能直接用于有任意重复值的一般 LCS。

A.6 实数二分与三分的前提

实数二分可固定迭代次数，避免直接比较浮点相等。例如连续函数在区间两端异号，可以保留仍有变号的半区间来寻找一个根。端点恰好为根时先处理；两个端点同号不代表没有根，也不能保证可以用这一规则寻找全部根。三次方程若有多个根，应先按题目给定性质隔离各根。

三分用于已知单峰或单谷的目标函数。对单峰求最大值，比较两个三等分点，舍弃不可能包含峰值的一侧；若函数存在多个峰，这个依据就不成立。分数规划、爬山和模拟退火还各有额外假设，不能用“都在搜索答案”把它们当作二分的替代模板。基础前三题训练应先把可证明的单调模型掌握牢固。

A.7 可删堆的延迟删除

优先队列通常只支持删除堆顶。若外部能保证删除请求对应一个当前存在的元素，可维护两个同类型堆：A 保存所有插入值，B 保存待删除值。每次取有效堆顶前，若两堆顶相等就同时弹出，直到不同或有堆为空。

重复值应按次数插入删除堆，不是用布尔标记；还要维护逻辑元素数，以区分真实为空与仍有待清理的节点。延迟删除让每个入堆元素最多被清理一次，但实现前应先判断 multiset 是否已经足够简单可靠。

A.8 字符串匹配的线性预处理

直接在每个起点比较模式串的复杂度最坏为 $O(nm)$，在小规模下完全可用。规模较大时，可用 KMP 的前缀函数维护“已匹配前缀还能退到多长”。设 pi[i] 为模式串前缀 0 到 i 的最长相等真前后缀长度：

```cpp
vector<int> prefix_function(const string &s) {
    vector<int> pi(s.size());
    for (int i = 1; i < (int)s.size(); ++i) {
        int j = pi[i-1];
        while (j > 0 && s[i] != s[j]) j = pi[j-1];
        if (s[i] == s[j]) ++j;
        pi[i] = j;
    }
    return pi;
}
```

对非空模式 p 扫描文本时，用同样的回退规则更新匹配长度 j；当 j 等于模式长度，记录一次匹配并回退到 pi[j-1]，从而允许重叠匹配。由于指针前进与回退的总量受线性界约束，预处理和匹配合计 $O(n+m)$。空模式的匹配规则应另行规定，不能直接访问 p[0]。

A.9 附录练习与解答

练习 1。数量 10 如何拆包？**解答**。1、2、4、3，和为 10，能表示 0 到 10 的每个件数。

练习 2。两个排列 $[3,1,2]$ 与 $[1,3,2]$ 的 LCS 长度是多少？**解答**。第一排列位置为 pos[3]=1、pos[1]=2、pos[2]=3；第二个映射为 $[2,1,3]$，LIS 长度 2，所以 LCS 长度 2。

练习 3。模式 aaa 在文本 aaaa 中出现几次？**解答**。2 次，起点 0 和 1。记录第一次后应保留可重叠的前后缀，而不是把匹配长度清成 0。

B 资料来源与继续练习

本讲义依据以下原始材料重新编排知识体系，补充了自包含教学例题、解答和参考程序。原课件中的重点概念问题已在正文采用修正后的表述，包括 unique 语义、DP 状态信息、背包初始化、BFS 归类和计数背包撤销。

原始文件	页数	本讲义对应内容
历年前三题考点梳理.pdf	5	题型安排、基础模拟、综合模拟选题方向。
练习题与真题.pdf	2	继续训练入口，仓库中另存链接索引。
stl.pptx	42	第 3–4 章与可删堆。
前缀和、差分、二分、双指针.pptx	34	第 5–6 章及搜索答案的选读内容。
搜索、剪枝、暴力.pptx	13	第 8 章与模板依赖展开。
dp 入门.pptx	18	第 9 章、LIS 与 LCS 扩展。
动态规划之背包问题.pptx	53	第 10 章与背包提高专题。

基础题进一步练习可选：排序 P1177、去重 P1059、前缀和 P8218、差分 P2367、二分 P2249、切木条 P2440、全排列 P1706、马的遍历 P1443、LIS B3637、最大子段和 P1115、01 背包 P1048、完全背包 P1616、分组背包 P1757。这些题可能有与教学题不同的输入、限制和输出要求，应重新阅读原题。

提高题可选：P2678、P1219、P3956、P1776、P1507、P1064、P1439、P4141。其中 P4141 的目标是删除指定物品后的恰好装满方案数末位数字，不是最大价值；P1439 的较大数据需要利用排列性质，普通二维 LCS 不能覆盖全部规模。

接口语义可查阅 C++ 工作草案的sort 条目和unique 条目。平台原题和标准文档是进一步核对的依据，本讲义正文的教学例题与解答已经提供独立学习所需的规则和算法。